

MySQL Backup Schedule with cron

To schedule a mysqldump in cron, you should **never** put the password directly in the crontab command (e.g., -p12345). Doing so makes the password visible to anyone running ps aux on the server and stores it in your shell history.

The secure, modern way to handle this is using a **MySQL Option File**.

1. Create a Secure Option File

Create a file specifically for the backup user. By convention, this is often named .my.cnf, but for a cron job, you can name it something specific like **/home/youruser/.backup.cnf**.

a. Create the file:

```
nano ~/.backup.cnf
```

b. Add your credentials:

```
[client]
user=backup_user
password=your_secure_password
```

c. Restrict Permissions (Crucial):

MySQL will ignore this file if it is "world-readable." It must be accessible only by your user.

```
chmod 600 ~/.backup.cnf
```

2. Test the Connection

Before putting it in cron, verify that the command works without asking for a password:

```
mysqldump --defaults-extra-file=~/.backup.cnf all_databases > test_backup.sql
```

3. Set up the Cron Job

Now you can safely add this to your crontab.

a. Open crontab:

```
crontab -e
```

b. Add the backup schedule:

This example runs every night at 2:00 AM, creates a file with a timestamp, and uses the config file for the password.

```
0 2 * * * /usr/bin/mysqldump --defaults-extra-file=/home/youruser/.backup.cnf --all-databases | gzip >
/home/youruser/backups/db_backup_$(date +%Y%m%d).sql.gz
```

Key Components of the Cron Command

Component	Purpose
<code>/usr/bin/mysqldump</code>	Always use the full path in cron jobs to avoid "command not found" errors.
<code>--defaults-extra-file</code>	Tells MySQL to read the password from your secure file.
<code>\$(date +%Y%m%d)</code>	Appends the current date to the filename.
<code>\%</code>	In cron, percent signs must be escaped with a backslash.
<code>gzip</code>	Compresses the backup to save disk space.

Alternative: Using `mysql_config_editor`

If you prefer not to have a plain-text file at all, you can use the `mysql_config_editor` tool to create an encrypted login file.

a. Create the login path:

```
mysql_config_editor set --login-path=backup_auth --host=localhost --user=backup_user --password
```

b. Use it in Cron:

```
0 2 * * * /usr/bin/mysqldump --login-path=backup_auth --all-databases > backup.sql
```

Add a simple "auto-cleanup" logic to your cron job.

This ensures you always have a rolling window of backups (e.g., 30 days) without manual intervention.

The Combined Cron Job (Backup + Cleanup)

You can chain these commands together in your crontab -e. This script will first create the compressed backup, and then find and delete any files in that folder older than 30 days.

```
0 2 * * * /usr/bin/mysqldump --defaults-extra-file=/home/youruser/.backup.cnf --all-databases | gzip > /home/youruser/backups/db_$(date +%F).sql.gz && find /home/youruser/backups/ -name "*.sql.gz" -type f -mtime +30 -delete
```

Breaking Down the Cleanup Command

- **find /path/to/backups/**: Looks in your backup directory.
- **-name "*.sql.gz"**: Only targets your backup files (prevents accidental deletion of other files).
- **-type f**: Ensures it only looks at **files**, not folders.
- **-mtime +30**: Matches files whose "modification time" is **more than 30 days** ago.
- **-delete**: Removes the matches.

Best Practice: Creating a Dedicated Backup Script

As your needs grow, putting long commands directly into crontab can get messy. It is often cleaner to create a small shell script.

1. Create the script:

```
nano ~/db_backup.sh
```

2. Paste this logic:

```
#!/bin/bash
BACKUP_DIR="/home/youruser/backups"
CONF_FILE="/home/youruser/.backup.cnf"
DATE=$(date +%Y-%m-%d)

# 1. Create the backup
/usr/bin/mysqldump --defaults-extra-file=$CONF_FILE --all-databases | gzip >
$BACKUP_DIR/db_$DATE.sql.gz

# 2. Delete backups older than 30 days
find $BACKUP_DIR -name "*.sql.gz" -mtime +30 -exec rm {} \;
```

3. Make it executable:

```
chmod +x ~/db_backup.sh
```

4. Update Cron:

```
0 2 * * * /home/youruser/db_backup.sh
```

Security Reminder

Ensure your /home/youruser/backups directory has restricted permissions so other users on the Ubuntu server cannot download your SQL dumps:

```
chmod 700 /home/youruser/backups
```